

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a21302bc3f7aa4305.jsonl`  
- SHA-256: `ed51cb552abc6c636c7c5b91f67bd4b91a016d7976c173926ee7e295ec50afc1`  
- Source modified: `2026-07-06T19:19:50+00:00`  
- Imported at: `2026-07-08T16:00:11+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:59.292Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy), src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (cross_adr): "Record failed policy decisions via delivery_attempts.last_error" conflicts with Track C's record_delivery contract (delivery outcome, not filter outcome)
severity medium | section Policy evaluation flow ("records the results via delivery_attempts.last_error (for failed items)")
summary: The ADR proposes the pipeline caller record rejected items via delivery_attempts.last_error. But a policy REJECTION means the item is never delivered, so no (content_item, destination) delivery is attempted ? Track C's record_delivery is about delivery transport success/failure and requires a destination_id and creates a row keyed (content_item_id, destination_id). Writing filter-rejection reasons there conflates "we chose not to deliver" with "delivery was attempted and failed."
why_it_matters: It misuses the just-merged Track C idempotency table. A rejected item has no destination to key on; forcing a delivery_attempts row with status=failed would poison retry logic (attempt_count/next_retry_at) and metrics that assume failed==transport failure. The explanation trace for rejections needs the deferred policy_decisions table, not delivery_attempts.
evidence: db.py record_delivery(*, content_item_id, destination_id, route_id, success, error): on failure sets status=DeliveryStatus.FAILED, increments attempt_count, writes last_error, keyed UNIQUE(content_item_id, destination_id). DeliveryStatus values are pending/delivered/failed/retrying ? all transport states (models.py). A Tier-1/2/3 rejection has no destination_id and is not a transport failure. ADR itself elsewhere says the real audit home is the deferred policy_decisions table.
rec: Drop the delivery_attempts.last_error path for policy rejections. State that rejected items are simply not delivered (and, when F6 lands, recorded in policy_decisions); delivery_attempts is written ONLY after a policy PASS when actual delivery is attempted (via record_delivery). This also keeps the pipeline placement ("before delivery") consistent with where record_delivery is called.

2. assistant (2026-07-06T19:19:02.730Z)

I'll verify this cross_adr finding by examining the ADR and the relevant code.

3. user (2026-07-06T19:19:04.606Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)            ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffprobe, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
```

```

 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```
```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase   | Scope                   | Dependencies                                                                  |
|---------|-------------------------|-------------------------------------------------------------------------------|
| ***F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler |
| ***F2** | Topic preferences       | `topics` config parsing, source title/caption keyword matching                |
| ***F3** | Source allowlist        | `allowlist` config, skip non-allowlisted sources                              |
| ***F4** | URL/domain reputation   | Domain blacklist, caption link extraction                                     |
| ***F5** | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                          |
| ***F6** | Policy decisions table  | Audit trail storage, explainability UI                                        |
| ***F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace                |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

## **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## **Alternatives considered**

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## **Open questions for the decision owner**

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

## **References**

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### **4. assistant (2026-07-06T19:19:08.277Z)**

Now let me examine the Track C code (record\_delivery, delivery\_attempts) and the models to verify the finding.

### **5. user (2026-07-06T19:19:10.285Z)**

c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)  
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E) (#39)

```

12d21b9 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C) (#37)
323ee5a feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B) (#35)
5fffc25b feat(identity): split users into accounts + telegram_sessions (ADR 0010, Track A) (#33)
---MODELS---
"""Pydantic data models for users, channels, videos, and filter config.

```

```

These models are the in-memory representation used by the repository layer
(`db.py`) and the filter engine (`filters/`). They are deliberately
decoupled from Telethon's own types so the filter engine stays pure and
unit-testable without a live Telegram connection.
"""

```

```

from __future__ import annotations

```

```

from datetime import UTC, datetime
from enum import Enum

```

```

from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator

```

```

def _utcnow() -> datetime:
 """Timezone-aware UTC 'now', used as a default factory."""
 return datetime.now(UTC)

```

```

class DeliveryMode(str, Enum):
 """How a matched video is delivered to the user.

 - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
 - ``LINK``: send a text message with a t.me link + metadata summary.
 - ``BOTH``: forward the video AND send the link message.
 """

 FORWARD = "forward"
 LINK = "link"
 BOTH = "both"

```

```

class Account(BaseModel):
 """A product identity ? the owner of config, channels, routes, and content.

 Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
 is the bridge used by the Bot API front-door (`from.id`) and by the
 legacy userbot path; it is nullable so future providers that don't have a
 Telegram id (e.g. WhatsApp) can still own an ``Account``.
 """

 model_config = ConfigDict(frozen=True)

 id: int = Field(description="Stable surrogate PK; the owner key for all downstream tables")
 telegram_id: int | None = Field(
 default=None,
 description="Telegram user id (bot from.id OR userbot account id). UNIQUE when set.",
)
 provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
 display_name: str | None = None
 created_at: datetime = Field(default_factory=_utcnow)

```

```

class TelegramSession(BaseModel):
 """An optional MTPROTO (Telethon) session linked to an :class:`Account`.

 A bot-only user has an ``Account`` row but no ``TelegramSession`` row;

```

```

that is what makes them structurally invisible to ``manager.run``
(ADR 0010). ``session_path`` is data-driven so the file-location
convention is explicit rather than derived from the PK at runtime.
"""

model_config = ConfigDict(frozen=True)

account_id: int = Field(description="FK ? accounts.id; PK (one session per account in v1)")
phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
enabled: bool = Field(
 default=True,
 description="If False, the manager skips this session (not the account)",
)
created_at: datetime = Field(default_factory=_utcnow)
last_login_at: datetime | None = None

class User(BaseModel):
 """Deprecated: legacy v1 model conflating identity + MTPROTO session.

 Retained as a backwards-compatible view over the new ``accounts`` +
 ``telegram_sessions`` tables (ADR 0010). New code should use
 :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
 maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
 and tests keep working without a flag day.
 """

 model_config = ConfigDict(frozen=True)

 id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy rows)")
 phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
 enabled: bool = Field(default=True, description="If False, the manager skips this session")
 created_at: datetime = Field(default_factory=_utcnow)

class Channel(BaseModel):
 """A channel/group a user is a member of. Populated lazily as messages arrive.

 Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
 code should use :class:`Source` directly; the ``db`` layer maps
 ``Channel`` ? ``Source`` so existing callers and tests keep working.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 user_id: int
 channel_id: int = Field(description="Telegram channel/chat id")
 title: str | None = None
 seen_first_at: datetime = Field(default_factory=_utcnow)

```

### --- provider-neutral domain models (ADR 0011, Track B) -----

#

**These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig`` shapes so the schema no longer leaks Telegram-specific ids into every future provider's code path. ``provider`` columns + provider-prefixed ``dedup\_key`` make cross-provider collisions structurally impossible. See ADR 0011 for the full rationale and the two decisions (Option 1A: tiered ``dedup\_key``;**



## Option 2A: ``policies`` generalizes ``filter\_config``).

```
class Source(BaseModel):
 """A configured place we ingest content from (ADR 0011).

 Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
 providers add their own source kinds). ``provider_source_id`` is the
 provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 account_id: int = Field(description="FK ? accounts.id (owner of the source)")
 provider: str = Field(default="telegram", description="Provider namespace")
 provider_source_id: str = Field(
 description="Provider-native source id (Telegram channel_id, ?)"
)
 title: str | None = None
 kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
 watched: bool = Field(
 default=False,
 description="ADR 0009 selective scanning: ingest from this source iff True",
)
 seen_first_at: datetime = Field(default_factory=_utcnow)
 last_seen_at: datetime | None = None

class ContentItem(BaseModel):
 """A normalized content record discovered from any :class:`Source` (ADR 0011).

 Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
 (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
 NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
 ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
 wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 account_id: int = Field(description="FK ? accounts.id")
 source_id: int = Field(description="FK ? sources.id")
 provider: str = Field(default="telegram", description="Provider namespace")
 provider_message_id: str = Field(
 description="Provider-native message id (Telegram message_id, ?)"
)
 content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
 text: str | None = Field(default=None, description="Caption / post text (nullable in Phase
1)")
 link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
 media_ref: str | None = Field(default=None, description="Provider media reference")
 document_id: str | None = Field(
 default=None,
 description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
)
 duration_s: int | None = None
 width: int | None = None
 height: int | None = None
 size_bytes: int | None = None
 mime_type: str | None = None
 dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per
account")
```

```

matched: bool = False
seen_at: datetime = Field(default_factory=_utcnow)

```

```

class Policy(BaseModel):
 """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).

 Generalizes the legacy per-account ``filter_config`` row so one account can
 own multiple policies (enables ADR 0009's per-route filter overrides) and so
 future enrichment/security stages grow into the same ``config_json`` column.
 Exactly one policy per account carries ``is_default=True``; the legacy
 ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 account_id: int = Field(description="FK ? accounts.id")
 name: str = Field(description="Policy name; 'default' for the account default policy")

```

## 6. assistant (2026-07-06T19:19:12.569Z)

Now let me look at the record\_delivery function and DeliveryStatus in detail.

## 7. user (2026-07-06T19:19:14.528Z)

```

src\tg_video_filter\models.py:262: destination_id: int = Field(description="FK ?
destinations.id")
src\tg_video_filter\models.py:268:class DeliveryStatus(str, Enum):
src\tg_video_filter\models.py:281: `` (content_item_id, destination_id)`` pair. On retry,
``attempt_count`` is
src\tg_video_filter\models.py:282: incremented and ``status``/``last_error`` are updated.
src\tg_video_filter\models.py:289: destination_id: int = Field(description="FK ?
destinations.id")
src\tg_video_filter\models.py:291: status: DeliveryStatus =
Field(default=DeliveryStatus.PENDING)
src\tg_video_filter\models.py:292: attempt_count: int = Field(default=0,
description="Incremented on each retry")
src\tg_video_filter\models.py:293: last_error: str | None = None
src\tg_video_filter\models.py:296: next_retry_at: datetime | None = None
src\tg_video_filter\delivery.py:131: destination_id: int | None = None,
src\tg_video_filter\delivery.py:139: If ``destination_id`` is provided AND
``write_delivery_attempt`` is True,
src\tg_video_filter\delivery.py:146: if write_delivery_attempt and destination_id is not
None:
src\tg_video_filter\delivery.py:150: destination_id=destination_id,
src\tg_video_filter\delivery.py:170: destination_id: int | None = None,
src\tg_video_filter\delivery.py:178: if write_delivery_attempt and destination_id is not
None:
src\tg_video_filter\delivery.py:182: destination_id=destination_id,
src\tg_video_filter\delivery.py:204: destination_id: int | None = None,
src\tg_video_filter\delivery.py:215: ``destination_id`` is the FK to ``destinations.id`` (ADR
0011 Track C).
src\tg_video_filter\delivery.py:220: client, link_text, conn, video_id, target,
destination_id=destination_id
src\tg_video_filter\delivery.py:223: # Both legs write to the same (content_item_id,
destination_id) row.
src\tg_video_filter\delivery.py:239: destination_id=destination_id,
src\tg_video_filter\delivery.py:248: destination_id=destination_id,
src\tg_video_filter\delivery.py:252: if destination_id is not None:
src\tg_video_filter\delivery.py:261: destination_id=destination_id,
src\tg_video_filter\delivery.py:279: client, message, conn, video_id, target,
destination_id=destination_id
src\tg_video_filter\db.py:160: destination_id INTEGER NOT NULL REFERENCES destinations(id),
src\tg_video_filter\db.py:164: UNIQUE (account_id, source_id, destination_id)

```

```

src\tg_video_filter\db.py:170: destination_id INTEGER NOT NULL REFERENCES destinations(id),
src\tg_video_filter\db.py:173: attempt_count INTEGER NOT NULL DEFAULT 0,
src\tg_video_filter\db.py:174: last_error TEXT,
src\tg_video_filter\db.py:177: next_retry_at TEXT,
src\tg_video_filter\db.py:178: UNIQUE (content_item_id, destination_id)
src\tg_video_filter\db.py:718: (account_id, source_id, policy_id, destination_id,
src\tg_video_filter\db.py:741: (content_item_id, destination_id, route_id, status,
attempt_count,
src\tg_video_filter\db.py:757: AND rr.destination_id = d.id
src\tg_video_filter\db.py:911: destination_id=row["destination_id"],
src\tg_video_filter\db.py:922: destination_id=row["destination_id"],
src\tg_video_filter\db.py:925: attempt_count=row["attempt_count"],
src\tg_video_filter\db.py:926: last_error=row["last_error"],
src\tg_video_filter\db.py:929: next_retry_at=row["next_retry_at"],
src\tg_video_filter\db.py:1339:async def record_delivery(
src\tg_video_filter\db.py:1343: destination_id: int,
src\tg_video_filter\db.py:1351: sets status=delivered; on failure, increments attempt_count
and records the
src\tg_video_filter\db.py:1354: status = DeliveryStatus.DELIVERED if success else
DeliveryStatus.FAILED
src\tg_video_filter\db.py:1357: (content_item_id, destination_id, route_id, status,
attempt_count,
src\tg_video_filter\db.py:1358: last_error, created_at, delivered_at)
src\tg_video_filter\db.py:1361: ON CONFLICT(content_item_id, destination_id) DO UPDATE
SET
src\tg_video_filter\db.py:1363: attempt_count=delivery_attempts.attempt_count + 1,
src\tg_video_filter\db.py:1364: last_error=excluded.last_error,
src\tg_video_filter\db.py:1369: destination_id,
src\tg_video_filter\db.py:1381: conn: aiosqlite.Connection, content_item_id: int,
destination_id: int
src\tg_video_filter\db.py:1386: "SELECT * FROM delivery_attempts WHERE content_item_id =
? AND destination_id = ?",
src\tg_video_filter\db.py:1387: (content_item_id, destination_id),

```

## 8. assistant (2026-07-06T19:19:20.378Z)

The finding is well-supported. Let me confirm the exact record\_delivery signature and DeliveryStatus enum values.

## 9. user (2026-07-06T19:19:20.931Z)

```

1339 async def record_delivery(
1340 conn: aiosqlite.Connection,
1341 *,
1342 content_item_id: int,
1343 destination_id: int,
1344 route_id: int | None = None,
1345 success: bool,
1346 error: str | None = None,
1347) -> None:
1348 """Record a delivery outcome for one (content_item, destination) pair.
1349
1350 This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
1351 sets status=delivered; on failure, increments attempt_count and records the
1352 error. Idempotent: repeated calls update the same row.
1353 """
1354 status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
1355 await conn.execute(
1356 """INSERT INTO delivery_attempts
1357 (content_item_id, destination_id, route_id, status, attempt_count,
1358 last_error, created_at, delivered_at)
1359 VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
1360 CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
1361 ON CONFLICT(content_item_id, destination_id) DO UPDATE SET

```

```

1362 status=excluded.status,
1363 attempt_count=delivery_attempts.attempt_count + 1,
1364 last_error=excluded.last_error,
1365 delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
1366 ELSE delivery_attempts.delivered_at END"",
1367 (
1368 content_item_id,
1369 destination_id,
1370 route_id,
1371 status.value,
1372 error,
1373 success,
1374 success,
1375),
1376)
1377 await conn.commit()
1378

```

#### 10. assistant (2026-07-06T19:19:23.387Z)

I have full confirmation of the code contract. Let me verify the DeliveryStatus enum values and check the ADR's exact wording once more.

#### 11. user (2026-07-06T19:19:24.146Z)

```

268 class DeliveryStatus(str, Enum):
269 """Per-destination delivery state, replacing the legacy binary ``forwarded``."""
270
271 PENDING = "pending"
272 DELIVERED = "delivered"
273 FAILED = "failed"
274 RETRYING = "retrying"
275
276
277 class DeliveryAttempt(BaseModel):
278 """Per-destination delivery record ? idempotent via UNIQUE index.
279
280 Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
281 ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
282 incremented and ``status``/``last_error`` are updated.
283 """
284
285 model_config = ConfigDict(frozen=True)
286
287 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
288 content_item_id: int = Field(description="FK ? content_items.id")
289 destination_id: int = Field(description="FK ? destinations.id")
290 route_id: int | None = Field(default=None, description="FK ? routing_rules.id
291 (nullable)")
292 status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
293 attempt_count: int = Field(default=0, description="Incremented on each retry")
294 last_error: str | None = None
295 created_at: datetime = Field(default_factory=_utcnow)
296 delivered_at: datetime | None = None
297 next_retry_at: datetime | None = None
298

```

#### 12. assistant (2026-07-06T19:19:39.262Z)

I now have full evidence. Let me evaluate the finding adversarially.

The ADR states in the "Policy evaluation flow" section: "The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table."

**\*\*Verifying the finding against the code contract:\*\***

1. `record_delivery` (db.py:1339)` requires `destination_id: int` (not optional)` and `content_item_id: int``, writing a row keyed `UNIQUE(content_item_id, destination_id)` (db.py:178)`. A policy rejection produces no delivery to any destination, so there is no `destination_id`` to key on.
2. `DeliveryStatus`` values (models.py:271-274) are pending/delivered/failed/retrying ? all transport states. `record_delivery`` sets `status=FAILED`` on `success=False` (db.py:1354)`, which semantically means a *\*delivery attempt\** failed, not that a filter rejected the item.
3. On the ON CONFLICT path (db.py:1363), `attempt_count`` increments ? this is retry accounting for transport. Writing a filter-rejection into this row would corrupt retry/metrics semantics.
4. The ADR itself acknowledges the real home is the deferred `policy_decisions`` table `(content_item_id, policy_id, stage_name, passed, reason, metadata_json, evaluated_at)`` ? which has no `destination_id``, confirming rejections don't fit the delivery model.

**\*\*Attempting to refute:\*\*** Could the ADR mean recording only rejections that happen *\*after\** a destination is chosen? No ? the pipeline runs "after dedup and before delivery," and Tier 1/2/3 rejections are fundamentally per-content (per-policy), not per-destination. The `delivery_attempts`` table cannot represent a rejection without a synthetic `destination_id`, and even then it would poison the idempotency/retry table. The "for failed items" phrasing clearly refers to policy-failed (rejected) items, conflating them with delivery-failed items. The finding holds.

**\*\*Severity calibration:\*\*** This is a genuine inaccuracy in a design contract ? it points the F1 implementer at a table whose schema and semantics cannot accommodate policy rejections, and would poison Track C's just-merged idempotency/retry logic if followed. It is not data-loss (no code exists yet) and the ADR offers the correct alternative (`policy_decisions`) as the OR-branch, so it is a real but self-mitigable design gap. Medium is correct.

### 13. user (2026-07-06T19:19:49.933Z)

Structured output provided successfully